

NanoGPT Speedrun: A Trail of Failures and One World Record

A Technical Retrospective

Alex Wa Andrew Xu Ben Xu

Yale University

PR #264 ([KellerJordan/modded-nanogpt#264](#))

Branch: [xsa-gated-layers](#)

May 6, 2026

Abstract

The NanoGPT speedrun [1] is a public benchmark that asks: how fast can eight GPUs drive a $\sim$ 124M-parameter transformer to 3.28 cross-entropy on the FineWeb validation set? Over roughly two weeks we surveyed forty experimental branches spanning optimizer, attention, activation, schedule, and architectural changes. The vast majority of them failed—some outright, some too marginal to survive an honest $n=10$ ablation. One idea, *Exclusive Self-Attention* (XSA) [2], when re-engineered into a per-(layer, head) learnable gate and broadened across all non-paired attention layers, yielded a clean wall-clock improvement over the master baseline at iso-loss. We submitted the result as PR #264. This document is the story of how we got there, including the dead ends and what each one taught us.

1 Background and goal

The benchmark is straightforward: train a GPT-2-(124M)-class language model on the FineWeb dataset until validation loss falls to or below **3.28**, and minimize wall-clock time on 8 NVIDIA H100 GPUs. The current public record on the `master` branch sits around 79 s, the product of nearly two years of community contributions: Muon optimization [3], FP8 `lm_head`, paired-head attention, value embeddings, sliding-window attention, ReLU-squared MLPs, $\sim$ thirty other tricks, and a great deal of systems engineering. Our hardware was an 8 $\times$ H200 node, so all wall-clock numbers in this report are H200-side. The architectural deltas we measured are hardware-independent; the wall-clock deltas (which decompose into step count $\times$ per-step time) require re-validation on H100 for the official ranking.

The structural difficulty is that almost every *plausible* idea has either already been tried or is ablated away by the careful design of the existing recipe. With seven steps’ wall-clock margin separating the bar from a sub-3.28 run, even a one-millisecond regression can wipe out an architectural win. Our bias going in: assume the master is locally optimal and that anything we add must *either* buy loss headroom we can spend on shorter schedules *or* cut milliseconds without raising loss. Anything else dies.

Baselines we measured (all on 8 $\times$ H200, $n=10$ each).

Configuration	Steps	Wall ($\mu \pm \sigma$)	Val loss ($\mu \pm \sigma$)
<code>master</code> (1440-step schedule)	1440	80.40 $\pm$ 0.11 s	3.27941 $\pm$ 0.00145
<code>master</code> cut to 1410 steps	1410	78.76 $\pm$ 0.05 s	3.28316 $\pm$ 0.00093

The second row exposes the core trade-off: a naive 30-step cut buys 1.64 s but lifts loss above the 3.28 bar. To win without raising step count, we needed an architectural change worth at least $\sim$ 0.004 nats of loss headroom—enough to absorb the 0.00375-nat penalty of the shorter schedule and still have margin. That target framed every experiment in this report.

2 Methodology

We organized work as a fork tree of branches off `master`, one experimental change per branch (e.g. `exp/xsa`, `exp/c-muon`, `exp/dyt`). Each branch was pre-screened with a single seed before committing to a real ablation.

Surviving branches were re-run at $n \geq 8$ with multiple seeds; for borderline candidates we followed the project’s small- n caution rule ($n=4$ std is artifactually tight) and ran $n = 10$ before claiming anything against **master**. We treat any decision below $n=8$ as conjecture.

We also held two hard rules. First, every architectural change had to begin from a configuration *bit-identical to master*—usually via zero-initialization of any new gate or scalar. This kept failure modes legible: if step-0 loss didn’t match the baseline, we had a bug, not a result. Second, we did not chase ideas whose only signal was qualitative (“this paper says it helps”); only loss-curve signal at the $n=10$ level gated submission.

3 What didn’t work

Forty branches were created. Most became negative results. We group them by the slice of the system they touched.

3.1 Optimizer changes

The Muon [3] side of the optimizer is heavily tuned, and most modifications regressed.

- **exp/c-muon**: cautious mask [4] (sign-agreement gate) on the polar-express output. Conceptually free—one bitmask, no extra matmul—but did not move loss at $n = 8$.
- **exp/adamuon**: AdaMuon [5] element-wise variance adaptation overlaid on NorMuon. The bf16 variance buffer added memory traffic; loss flat, wall slightly worse.
- **exp/per-head-normuon**: redistribute NorMuon’s per-axis EMA step-size budget across heads on the QK bank. No measurable signal.
- **exp/hyperball-adam**: pin `lm_head` Frobenius norm at its initial value (replacing weight decay). Mild loss regression.
- **exp/ademamix**: AdEMAMix [6] two-EMA Adam variant for the embedding/scalar bank. The slow EMA needs many more steps than this 1500-step regime allows; losing.
- **exp/dion**, **exp/scion-adam-leg**, **exp/schedule-free-adam**: each replaced or augmented an optimizer leg; none beat the existing Muon+NorMuon+Adam mix.
- **exp/sr-bf16-adam**: store Adam moments in bf16 with stochastic rounding [7]. Halves optimizer-state memory and reduces all-gather traffic. We saw a small wall-clock win but a ~ 0.001 -nat loss regression that ate the gain.
- **exp/dsv4-newton-schulz**: swap Polar Express for the DeepSeek-V4 hybrid 8+2 Newton-Schulz schedule [8]. Net loss regression at iso-step.

3.2 Schedule and curriculum

- **exp/adam-warmup**, **exp/adam-warmup-10**: short Adam-only LR warmup (50 / 10 steps). Inconsistent at $n = 8$.
- **exp/muon-lr-floor**: split Adam/Muon cooldown floors (0.15 / 0.80). No signal vs. master’s shared 0.15 floor.
- **exp/cwd-acceleration**: split Cautious Weight Decay into separate acceleration and decay coefficients. Flat.
- **exp/power-scheduler**: power-law cooldown [9] replacing the linear decay. Shape changes the time spent near peak LR but did not help in this regime.
- **exp/seq-length-curriculum** and **exp/seq-curriculum-batch-bump**: shorter `train_max_seq_len` on early stages, optionally with a stage-1 batch bump. The wall-clock gain we expected from cheaper varlen attention was eaten by recompiles and a slight loss regression.
- **exp/leaky-relu-iter-sweep**: `NUM_SCHEDULED_ITERATIONS` env override for sweeping. Infra change, not a result.

3.3 Attention and value-stream changes

- **exp/qk-gain**: per-(layer, head) learnable post-QK-norm gain. Identity-init, but the optimizer found no useful per-head temperature beyond YaRN’s existing scale.
- **exp/ssmax**: SSMax [10] log-context attention scaling. Theoretically composes with the YaRN window schedule; empirically flat.
- **exp/long-window-gqa**: GQA-by-averaging on long-window layers (3 and 10), 3:1 K/V $\rightarrow$ Q grouping. After fixing a bug where averaging happened *before* rotary/value-embedding mixing, the corrected version was loss-neutral but gave us no wall-clock either (averaged tensors still have full per-head shape).
- **exp/fox-gate**: FoX data-dependent forget-gate bias on long-window layers. Bf16-cast and doc/window mask issues; final version flat.
- **exp/selective-attention**: parameter-free selective attention [11] on layer 10 only. After fixing three correctness bugs (acausal cumsum direction, missing doc mask, missing window mask), the materialized (T, T) path made the layer too slow; loss signal alone could not justify the regression.
- **exp/attn-sink**: per-(layer, head) learnable attention sink (DeepSeek-V4 §2.3.3 [8]). Loss neutral.
- **exp/register-tokens**: 4 learnable BOS-position offsets [12]. Composes with the existing BOS-alignment recipe but added no loss headroom.
- **exp/rotating-dim-sweep**: parameterize the RoPE partial-rotation dim (32, 48, 64, 80, 96). The default $head_dim/2 = 64$ remained best.

3.4 Architectural and capacity changes

- **exp/dyt**: Dynamic Tanh [13] at pre-attn and pre-MLP norms. Flat to slightly negative.
- **exp/xielu**: xIELU activation [14] with per-layer learnable α_p, α_n , eventually with a fused Triton kernel. Same loss as ReLU² at slightly worse wall-clock.
- **exp/leaky-relu-spectral-init**: LeakyReLU(α)² MLP plus an overtone power-law spectral `lm_head` init. Two ideas in one branch; signal too noisy at $n = 8$ to justify either.
- **exp/hourglass-ffn**: shrink MLP inner dim from $4d$ to $2d$. Wall-clock improvement, loss regression, no net.
- **exp/progressive-depth**: gate top layers off early in training, phase in later. The compiled graph still ran the gated layers at full cost (`torch.compile` doesn’t dead-code-eliminate buffers), so we measured loss only; even there it was flat.
- **exp/hmid-injection**: zero-init mid-network state injection into later layers. Flat.
- **exp/per-token-resid-lambda**, **exp/backout-mlp-gate**: per-token sigmoid gates around residual lambdas / backout. Both no-op-init; both flat.
- **exp/mhc**: minimal $n_{hc} = 2$ manifold-constrained hyper-connections, zero-init via lane-0 dominance. Flat.
- **exp/mudd-residual**: per-token gated dense skips into layers 9 and 10 (residual channel). Not promising on its own—but, kept warm; it would resurface (§6).

3.5 Systems and infrastructure

- **exp/cuda-graphs**: capture inductor in `reduce-overhead` mode. Compile-time win, no end-to-end speedup once warm.
- **exp/lawa-wsm**: disable LR cooldown and merge checkpoints at final eval. Not robust at $n = 8$; a small fluke can flip the average.
- **exp/opt-overlap-investigation**: probe pipeline overlap between Muon and Adam steps. Diagnostic only.

The cumulative lesson from these failures: the master is tighter than it looks. About a third of these branches “should have” helped on paper, and several were direct ports of papers reporting big gains at larger scale. Most papers’ wins evaporate when the harness is already running 1500 steps with a heavily tuned schedule and a 9 ms/step compute budget.

4 What did work: the XSA path

The one survivor came from *Exclusive Self-Attention* (XSA) [2]. The premise: when the attention output y is read back into the residual stream, it includes a component aligned with the current token’s own value vector v , which the token already knows. XSA explicitly removes that self-aligned projection:

$$\hat{v} = v/\|v\|, \quad y \leftarrow y - \langle y, \hat{v} \rangle \hat{v}.$$

No new parameters, no extra matmul; just two element-wise vector ops on the FlashAttention output. That made it cheap enough to add to a path that already costs only ~ 54 ms/step.

First attempt (exp/xsa, layer set $\{7, 8, 10\}$). We applied XSA to the deepest non-paired attention layers, where the self-attention bias is most likely to dominate. Paired layers $\{0, 2, 5, 9\}$ reshape v across head pairs (head-projection math doesn’t line up with XSA’s per-head subtraction), and layer 6 is MLP-only. This first cut was bit-identical to master at step 0 only by virtue of being a deterministic op—there was no learnable strength. Result at $n = 8$: a small but real loss improvement, but no wall-clock to spend at iso-step. The result was directional, not closing.

Step 2: make it learnable (exp/xsa-gated). We replaced the unconditional subtraction with a learnable per-(layer, head) gate:

$$y \leftarrow y - \tanh(\alpha_{\ell,h}) \cdot \langle y, \hat{v} \rangle \hat{v}, \quad \alpha \in \mathbb{R}^{L_{\text{xsa}} \times H}, \alpha \equiv 0 \text{ at init.}$$

Zero-init means $\tanh(0) = 0$, so the model is bit-identical to master at step 0. The gate can grow to +1 (full XSA), shrink toward 0 (no-op), or even go slightly negative (mild self-amplification, if the optimizer wants it). The strength sits in the Adam bank with a modest LR multiplier and no weight decay.

Diagnostics (exp/xsa-gated-diag). We logged final $\tanh(\alpha_{\ell,h})$ per (layer, head) at end of training. A representative run:

Layer	$\tanh(\alpha)$ per head $h \in \{0, \dots, 5\}$	Note
7	[+0.876, +0.769, +0.837, +0.918, +0.961, +0.795]	saturated near +1
8	[+0.469, +0.084, +0.171, +0.562, +0.408, +0.379]	mid-range, varied
10	[+0.384, +0.443, +0.635, +0.492, +0.396, +0.622]	mid-range

$|\alpha|$ max 0.961, mean 0.567, all 18 (layer, head) pairs “active” ($|\tanh \alpha| > 0.05$). The optimizer was using the gate, not zeroing it—the strongest signal that the operation, at least on these layers, was load-bearing.

Negative ablation: pruning layer 8 (exp/xsa-gated-prune-layer-8). The diagnostic showed layer 8’s $|\alpha|$ was the smallest. We tested removing it. No improvement vs. $\{7, 8, 10\}$. Lesson: small α does not mean “not used”—a 0.1 gate on a long-tail correction can still earn its place in a bandwidth-bound op.

Step 3: broaden to all non-paired layers (exp/xsa-gated-more-layers). Up to this point XSA lived on the last three non-paired layers. We extended it to *all* non-paired attention layers: $\{1, 3, 4, 7, 8, 10\}$. Six layers, twelve heads each, 72 new scalar parameters. At $n = 10$, this lifted the loss headroom enough to support a shorter schedule.

Step 4: spend the headroom (exp/xsa-gated-more-layers-fewer-steps). A 30-step cut on master alone over-shoots the 3.28 bar by 0.00375 nats; the broader gated XSA gave us about 0.0008 nats of headroom relative to master. We cut `num_scheduled_iterations` from 1440 to 1410 (with the existing 40 extension iters held fixed at 1450 total runtime steps). Wall-clock dropped to 79.85 s; loss came in at 3.27865, comfortably under the bar.

5 The world record submission

The full $n = 10$ ablation, all on $8\times\text{H200}$, is:

Configuration	Runs	Steps	Wall (s, $\mu \pm \sigma$)	Δ wall	Val loss ($\mu \pm \sigma$)
master-1440 (baseline)	10	1440	80.400 ± 0.105	—	3.27941 ± 0.00145
master-1410 (naive cut)	10	1410	78.760 ± 0.046	-1.640	3.28316 ± 0.00093
Gated XSA, 1410 (this PR)	10	1410	79.846 ± 0.206	-0.554	3.27865 ± 0.00105

The PR is the third row. Loss improvement vs. **master-1440** is -0.00076 nats (Welch $p \approx 0.0014$, $n = 10$); wall improvement is -0.554 s. The naive step cut is faster but fails the 3.28 bar by 0.00316 nats with $4\text{-}\sigma$ confidence. The architectural change is what unlocks the cut.

Why the cost is small. XSA introduces no new matmul: $\langle y, \hat{v} \rangle$ is a per-head dot product on $(\mathbf{B}, \mathbf{T}, \mathbf{H}, \mathbf{D})$ tensors, the normalization is a per-token rsqrt-multiply, and the gated subtraction is a single fused element-wise op. The added overhead is bandwidth-bound; on H200 we measured roughly 0.6 ms/step amortized. On H100, the lower bf16-activation bandwidth narrows the margin but the per-step overhead remains well under 1 ms , far less than the $30 \times 53.5 = 1.6\text{ s}$ saved by the schedule cut.

Why zero-init mattered. We had been burned on previous branches by “no-op init” that wasn’t bit-identical because of bf16 rounding or a missed cast. Forcing $\alpha \equiv 0$ kept the step-0 forward, the entire Triton kernel call sequence, and the FP8 `lm_head` inputs identical to master. That meant any non-zero loss delta after step 0 had to come from the gate—we never had to argue away an integration bug.

Where the alphas grew. Across 10 final runs, layer 7 saturated near $+0.9$ on all six heads, layer 10 stabilized in the mid-range, and layer 8 was head-heterogeneous (some heads at $+0.5$, others near zero). We initially read low- $|\alpha|$ heads as “vestigial,” but the prune-layer-8 ablation contradicted that interpretation: the gate is monotonic information; even a $|\alpha| = 0.1$ row can be net-helpful when the operation is essentially free.

6 What we tried after the record

Two follow-up branches stacked further changes on top of the world-record configuration. They are documented for completeness; neither has yet been validated to the $n=10$ standard required for an official submission.

exp/xsa-mudd: stack MUDD dense skips on Gated XSA. We ported a multiway-dynamic-dense-connection (MUDD) [15] block from the Lisennlp fork: layer 9 emits a v_{mudd} tensor and a residual delta with a self-ref fuse $(1 + m_{r,9})x$, plus six layer-10 scalar gates and a value-embedding gate. At step 0 the MUDD block reduces to the master scalar inits. XSA fires after $v+ = v_{\text{mudd}}$, so on layer 10 it operates on the MUDD-modified V . At 1405 scheduled iters + 10 extension iters ($= 1415$ steps), $n = 8$ runs landed at **val 3.27390 ± 0.00130** in roughly **78.4 s** — 0.005 nats below the 3.28 bar at a noticeably faster wall-clock than the world-record config. Promising, but not yet $n=10$ -validated; per our small- n rule, we treat this as a candidate, not a record.

exp/xsa-mudd-1360: spend the MUDD headroom on more steps. Cut to 1360 scheduled + 10 extension iters ($= 1370$ steps), targeting $\sim 2.5\text{ s}$ of wall savings. Across 10 runs the mean validation loss drifted up to roughly 3.2797 —right at the bar. The variance was wide enough that, conservatively, this cut went too far. The right operating point is somewhere between 1370 and 1415 steps; the next ablation will be a step sweep at small granularity.

Other branches that may have legs.

- `exp/xsa` (the deterministic, no-gate, three-layer original) is the closest precursor and a reasonable simpler-than-PR alternative if the gate ever proves to add reviewer friction.
- `exp/xsa-gated` on the original $\{7, 8, 10\}$ subset is loss-positive but wall-clock-flat; it is what *would* have been the submission if we had stopped before broadening to all non-paired layers.

7 What we wish we’d known earlier

A few patterns recur across the failed branches and we would have moved faster with them written down on day one.

1. **Iso-init or iso-nothing.** Every “no-op init” that wasn’t bit-identical to master either hid bugs or made it impossible to attribute later loss deltas. The XSA gate worked partly because $\tanh(0)$ is exactly 0 in any precision.
2. **Cheap-to-disable beats clever-default.** A learnable scalar at zero is reversible. A new optimizer leg, a new compile shape, a new kernel, are not. We over-spent on branches that committed to a structure before measuring whether the optimizer wanted it.
3. **Small- n lies confidently.** Several branches looked positive at $n = 4$ and reverted at $n = 10$. We adopted “no record claim under $n = 10$ ” midway through and should have done so on day one.
4. **Frontier-paper translation isn’t free.** DeepSeek-V4-style ideas (attention sink, MHC, NS schedule) don’t translate cleanly to a 1500-step, 124M-parameter regime. The speedrun’s curriculum and FP8-`lm_head` interactions kill many otherwise-good ideas.
5. **Gates beat replacements.** Of the architectural changes that ever showed positive signal in our search, all of them were additive gates with reversible defaults. We did not see a single *replacement* (DyT for RMSNorm, xIELU for ReLU², GQA for MHA, hourglass for 4d FFN) survive an $n=10$ ablation.

8 Conclusion

The headline result is a single PR (#264) that saves 0.554s on the $8\times H200$ speedrun by trading a learnable per-(layer, head) Exclusive Self-Attention gate for a 30-step shorter schedule, while remaining 0.0008 nats below the 3.28 bar. Beneath that single PR are roughly forty branches, of which four made it to a clean ablation and only one crossed the bar. The XSA-MUDD continuation looks like the next plausible record; getting it to $n = 10$ is the most direct path from where we are. The rest of the failed branches are documented in the repository under `exp/*`; collectively they form a moderately reliable map of which directions *don’t* pay off in this particular regime.

Reproducing the result. The PR sits on `xsa-gated-layers` at commit 89636ed. It adds ~ 30 lines to `train.gpt.py` (the gate parameter, a normalize-and-subtract on FlashAttention’s output, registration in the Adam bank) and changes one constant: `num_scheduled_iterations` from 1440 to 1410. Re-validation on H100 is the only outstanding step before the result can be claimed officially.

References

- [1] Keller Jordan et al. *Modded-NanoGPT: speedrunning GPT-2-(124M) to 3.28 val loss*. <https://github.com/KellerJordan/modded-nanogpt>. 2024–2026.
- [2] Yuxin Zhai et al. “Exclusive Self-Attention: Removing Self-Aligned Components from Attention Outputs”. In: *arXiv preprint arXiv:2603.09078* (2025).
- [3] Keller Jordan. “Muon: An Optimizer for the Hidden Layers of Neural Networks”. In: (2024).
- [4] “Cautious Optimizers: Improving Training with One Line of Code”. In: *arXiv preprint arXiv:2411.16085* (2024).

- [5] “AdaMuon: Element-wise Variance Adaptation for Spectral Update Optimizers”. In: *arXiv preprint arXiv:2507.11005* (2025).
- [6] “AdEMAMix: Mixing Two EMAs of Gradients for Improved Optimization”. In: (2024).
- [7] “Stochastic Rounding for AdamW in bf16”. In: *arXiv preprint arXiv:2502.20566* (2025).
- [8] DeepSeek-AI. *DeepSeek-V4 Technical Report*. 2026.
- [9] “Power Scheduler: A Batch-Size-Invariant Learning-Rate Scheduler”. In: *arXiv preprint arXiv:2408.13359* (2024).
- [10] “Scalable-Softmax Is Superior for Attention”. In: *arXiv preprint arXiv:2501.19399* (2025).
- [11] “Selective Attention Improves Transformer”. In: *arXiv preprint arXiv:2410.02703* (2024).
- [12] “Vision Transformers Need Registers”. In: *arXiv preprint arXiv:2309.16588* (2023).
- [13] “Dynamic Tanh: Replacing Layer-Norm with a Cheaper Pre-Activation”. In: *arXiv preprint arXiv:2503.10622* (2025).
- [14] NVIDIA. “xIELU: Trainable Piecewise Activation”. In: *arXiv preprint arXiv:2411.13010* (2024).
- [15] et al. Lin. “MUDD: Multiway Dynamic Dense Connections for Transformer Residual Streams”. In: *arXiv preprint* (2025). Implementation referenced from <https://github.com/Lisennlp/modded-nanogpt> fork.